

Cleaning Strategy

The goal was to make raw CSV data **clean, consistent, and analytics-ready**. Key steps:

1. Initial Load

- Read CSV with `inferSchema=False` so all columns start as `StringType` (avoids load errors due to mixed formats).

2. Text Normalization

- Trim leading/trailing spaces in city, category, product using `trim()`.
- Convert to **Proper Case** using `initcap()` for consistency.

3. Amount Column

- Remove commas using `regexp_replace()`.
- Replace blanks and invalid values with `NULL`.
- Validate with regex before casting to `IntegerType` to prevent crashes.

4. Date Column

- Support multiple formats (`yyyy-MM-dd`, `dd/MM/yyyy`, `yyyy/MM/dd`) using `try_to_timestamp()` + `coalesce()`.
- Create `order_date_clean` as `DateType` while keeping original for auditing.

5. Status & Deduplication

- Normalize status and filter only "Completed".
- Remove duplicates by `order_id` using `dropDuplicates()`.

6. Data Quality Checks

- Count invalid amounts and dates.
- Validate row counts after each stage.

Performance Strategy

Optimizations applied to handle large-scale data efficiently:

1. Partition Awareness

- Checked `getNumPartitions()` and tuned `spark.sql.shuffle.partitions` for balanced parallelism.
- Repartitioned by city before writing partitioned Parquet and before heavy aggregations.

2. Caching

- Cached `completed_df` since it was reused across multiple queries (aggregations, joins).
- Verified performance improvement by comparing execution times before and after caching.

3. Broadcast Joins

- Used `broadcast()` for small lookup tables (city → region) to avoid shuffles and leverage BroadcastHashJoin.

4. Columnar Storage

- Wrote cleaned data to **Parquet** (partitioned by city) and aggregates to **ORC** for compression and predicate pushdown.
- Compared size and read performance against CSV.

5. Explain Plans

- Used `explain(True)` to identify shuffle boundaries and confirm optimizations like broadcast join and partitioning.

Debugging Learnings

Challenges and solutions during pipeline development:

1. Date Parsing Errors

- Initial use of `to_date()` threw `DateTimeException` for mixed formats.
- Fixed by switching to `try_to_timestamp()` for tolerant parsing and `coalesce()` for multiple formats.

2. Invalid Amounts

- Direct casting caused pipeline crashes.
- Solved by regex validation and safe casting with `when()` + `otherwise(NULL)`.

3. Duplicate Handling

- `dropDuplicates()` works but is non-deterministic.
- Learned to use `Window` + `row_number()` for deterministic deduplication when needed.

4. Performance Bottlenecks

- Multiple aggregations on the same `DataFrame` were slow.
- Resolved by caching and tuning shuffle partitions.

5. Storage Validation

- Verified schema and row counts after writing Parquet/ORC to ensure no data loss.
- Compared sizes and read times to confirm columnar formats outperform CSV.